



Università
di Catania

Dipartimento di
Ingegneria Elettrica Elettronica e
Informatica

Advanced Programming Languages

C++: exceptions; containers

Docente:
Marco Grassia
marco.grassia@unict.it

A.A. 2025/2026

Exceptions

Advanced Programming Languages: C++

Exceptions

- **An exception is an error or an unexpected condition that occurs during program execution**, such as a division by zero
- **Exception handling is the mechanism that allows a program to detect and respond to such events** without crashing
- Exceptions **separate normal program flow from error-handling logic**, improving code clarity and robustness
- **An exception is usually *thrown* (or raised) when something goes wrong. Some code might be ready to *catch* it and deal with it**
- **If an exception is not caught, it will propagate further, ultimately causing the program to terminate**

Exceptions in C++:

try; throw; catch

- ***throw*** is used to **raise an exception** when the code encounters a problem. It **creates and sends an exception object**
- ***try*** is **used to wrap a risky code block** that might raise an exception, and is followed by one (or more) ***catch*** statements
- ***catch*** **declares an exception handling code block** for a specific exception type. If no usable catch block is found, `std::terminate` is invoked and the program exits

```
try {  
    // protected risky code  
} catch (ExceptionName e1) {  
    // catch block  
} catch (ExceptionName e2) {  
    // catch block  
} catch (ExceptionName eN ) {  
    // catch block  
}
```

Exceptions in C++

- It is recommended to **catch exceptions by constant reference**, not by value
- This **avoids unnecessary copies** and **preserves polymorphic behavior**
- **Object slicing** occurs when a derived class object is assigned or passed by value to a base class type, causing the derived part to be “sliced off”

```
using std::cout, std::endl;
using std::out_of_range;

char myArray[10];
try{
    for(int n=0; n <=10; n++){
        if (n>9) throw out_of_range("invalid index");
        myArray[n] = 'm';
    }
}
// Bad practice: catch by value, object slicing
catch(out_of_range e) { }
// Good practice: catch by reference to avoid object slicing
catch(const out_of_range& e){
    cout << "Exception: " << e.what() << endl;
}
```

Exceptions in C++:

throwable types

- Technically, **any data type can be thrown** **but** that's a **bad practice**, since they lack structure and don't provide meaningful error information
- All **thrown exceptions should derive from *std::exception*** that **provides** the virtual method
`virtual const char* what() const noexcept;`

```
throw 1;
```

```
throw "error";
```

```
throw 2.314f;
```

```
using std::cout, std::endl;
```

```
char myArray[10];
```

```
try{  
    for(int n=0; n <=10; n++){  
        if (n>9) throw "invalid index";  
        myArray[n] = 'm';  
    }  
}  
catch (char *s) {  
    cout << "Exception: " << s << endl;  
}
```

Exceptions in C++

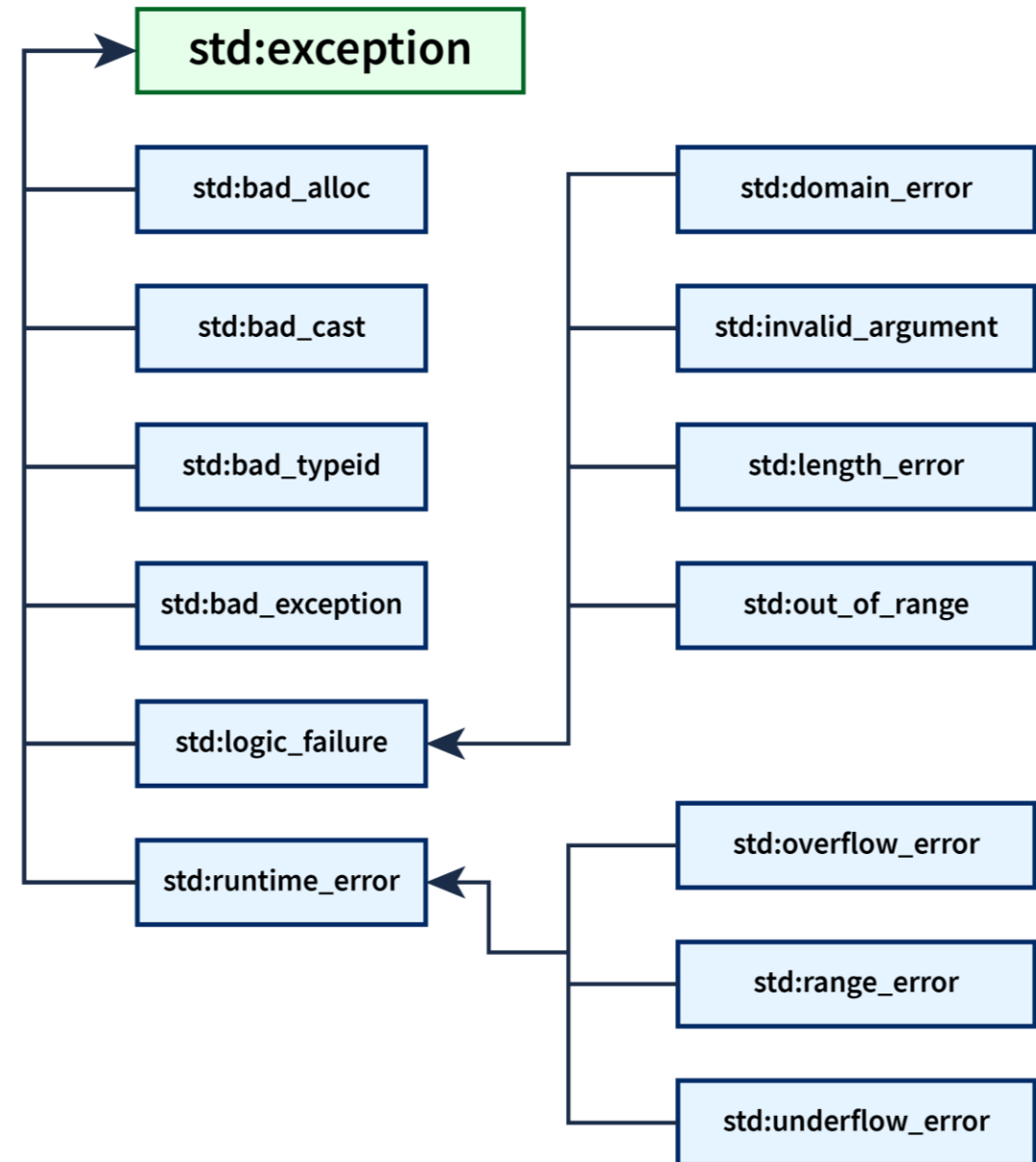
throwable types: *std::exception*

- **All standard exceptions in C++ inherit from `std::exception`**, and all thrown exceptions should derive from it
- ***std::exception* provides** the virtual method
`virtual const char* what() const noexcept;`

```
void risky() {  
    throw std::runtime_error("Critical error");  
}  
  
int main() {  
    try {  
        risky();  
    } catch (const std::exception& e) {  
        std::cerr << "Caught: " << e.what() << '\n';  
    }  
}
```

C++ standard exceptions

C++ provides a set of standard exceptions that are defined in `std::exception` and that can be used within our programs



Exceptions in C++:

default catch

- `catch (...)` is a **catch-all handler** that **matches any exception type**
- Useful for logging or cleanup when the specific type is unknown
- Avoid overuse: it can hide programming errors

```
try {  
    throw std::runtime_error("Error occurred");  
}  
catch (...) {  
    std::cerr << "Generic exception caught\n";  
}
```

Exceptions in C++:

Re-throwing

- A bare throw; inside a catch block **rethrows the current exception**
- Preserves the original exception type and data

```
try {  
    throw std::runtime_error("Error occurred");  
}  
catch (...) {  
    std::cerr << "Generic exception caught\n";  
    throw; // rethrows the same exception  
}
```

Exceptions in C++: where's the finally block?!

- **C++ does NOT have finally blocks**
- Instead, it uses **RAII (Resource Acquisition Is Initialization)**:
 - Objects manage resources in their destructors
 - Destructors run automatically when leaving scope, even during exceptions
- This ensures cleanup without explicit finally

```
try {  
    throw std::runtime_error("Error occurred");  
}  
catch (...) {  
    std::cerr << "Generic exception caught\n";  
    throw; // rethrows the same exception  
}
```

Exceptions in C++:

Examples of instructions that *throw*

- **Memory Allocation**

- `new` → throws `std::bad_alloc` on failure
- `std::vector::resize` or `push_back` → may throw `std::bad_alloc`

- **Standard Library Operations**

- `std::string::at()` → throws `std::out_of_range` if index invalid
- `std::vector::at()` → same behavior
- `std::map::at()` → throws `std::out_of_range` if key not found

Exceptions in C++:

Examples of instructions that *throw*

- **Type Casting**

- `dynamic_cast<Derived&>(base)` → throws `std::bad_cast` if cast fails

- **I/O Streams**

- `std::cin >> x` → can set `failbit` or `badbit`; if `exceptions()` are enabled, throws `std::ios_base::failure`

- **Filesystem**

- `std::filesystem::remove(path)` → throws `std::filesystem::filesystem_error` on failure

C++ standard exceptions

- `bad_typeid`
- `bad_cast`
 - `bad_any_cast(C++17)`
- `bad_weak_ptr(C++11)`
- `bad_function_call(C++11)`
- `bad_alloc`
 - `bad_array_new_length(C++11)`
- `bad_exception`
- `ios_base::failure(until C++11)`
- `bad_variant_access(C++17)`

C++ standard exceptions

- `logic_error`
 - `invalid_argument`
 - `domain_error`
 - `length_error`
 - `out_of_range`
 - `future_error`(C++11)
- `bad_optional_access`(C++17)
- `runtime_error`
 - `range_error`
 - `overflow_error`
 - `underflow_error`
 - `regex_error`(C++11)
 - `nonexistent_local_time`(C++20)
 - `ambiguous_local_time`(C++20)
 - `tx_exception`(TM TS)
 - `system_error`(C++11)
 - `ios_base::failure`(C++11)
 - `filesystem::filesystem_error`(C++17)

C++ exceptions:

noexcept

- `noexcept` declares that a function **will not throw exceptions**
- It **improves optimizations** (e.g., inlining, move operations), **and communicates intent to callers so that they don't need to handle exceptions**
- If a `noexcept` function **does throw**, `std::terminate()` is called and program is terminated
- The compiler does **not check** for thrown exceptions **at compile time**, the **enforcement** happens **at runtime**

```
void f() noexcept;

void f() noexcept {
    throw std::runtime_error("Error");
    // Program terminates
}
```

C++ exceptions: custom exceptions

- You can define exceptions by **deriving from `std::exception`** or its subclasses
- Remember to **implement constructors and override `what()`** to provide a custom message
- Always mark `what()` as **`noexcept`** (required by `std::exception`)
- Use `explicit` for constructors to avoid unintended conversions

```
class MyException: public std::exception {
    std::string message_;
public:
    MyException() :
        MyException("Default MyException message") {}

    explicit MyException(const std::string& msg):
        message_(msg) {}

    const char* what() const noexcept override {
        return message_.c_str();
    }
};
```

Standard library (STD)

Advanced Programming Languages: C++

The C++ STandarD Library (STD)

- The standard library is a **collection of classes and functions** built into C++, meant **to improve productivity and portability** by offering **ready-to-use, highly optimized tools**
- Supports generic programming and has documented complexity for predictable performance
- It provides essential components for:
 - **Containers and data structures** (e.g., vector, map, set)
 - **Algorithms** (e.g., sort, find)
 - **Input/Output** (e.g., iostream)
 - **Utilities** (e.g., std::pair, std::tuple)
 - **...**

stdlib: containers

- **Sequence containers:** `vector`, `deque`, `list`, `forward_list`, `array`
- **Associative (ordered):** `map`, `multimap`, `set`, `multiset`
(logarithmic ops)
- **Unordered (hash-based):** `unordered_map`, `unordered_set`
(avg $O(1)$ ops)
- **Adaptors :** `stack`, `queue`, `priority_queue` (restricted interfaces)

stdlib: containers

- For most workloads, **std::vector** is the default
 - it's **contiguous** (cache-friendly)
 - supports **amortized $O(1)$ push_back**
 - plays nicely with algorithms
- Reach for **ordered associative** containers (`std::map`, `std::set`) when you need **sorted keys** or **stable iterators**, accepting $O(\log N)$ lookups

stdlib containers:

`std::vector`

For most workloads, `std::vector` is the default

- it's **contiguous** (cache-friendly)
- index access is $O(1)$
- supports **amortized** $O(1)$ **push_back**
- plays nicely with algorithms

```
std::vector<int> v;  
// preallocate to avoid early reallocation  
v.reserve(4);  
  
v.push_back(1);  
v.push_back(2);  
  
// valid pointer to first element  
int *p = &v[0];  
// still within capacity →  
// no reallocation; 'p' remains valid  
v.push_back(3);  
// capacity now full  
v.push_back(4);  
// triggers reallocation →  
// 'p' becomes INVALID (new block)  
v.push_back(5);
```

stdlib containers:

`std::vector`

- **Automatically grows** (i.e., reallocates) **when capacity is reached** (actual policy is implementation defined, but typically grow by about **1.5× to 2×**)
- Use **`reserve(n)`** to pre-allocate and **reduce the number of full copies/moves**
- **On growth, pointers are invalidated**

```
std::vector<int> v;  
// preallocate to avoid early reallocation  
v.reserve(4);  
  
v.push_back(1);  
v.push_back(2);  
  
// valid pointer to first element  
int *p = &v[0];  
// still within capacity →  
//           no reallocation; 'p' remains valid  
v.push_back(3);  
// capacity now full  
v.push_back(4);  
// triggers reallocation →  
//           'p' becomes INVALID (new block)  
v.push_back(5);
```

stdlib containers:

`std::array`

- If you need **fixed-size contiguous stack storage**, `std::array<T, N>` avoids heap allocations
- Effectively stores **T[N] inline inside the object**
- Size **N** is part of the type
- Size $\approx N * \text{sizeof}(T)$ (plus possible padding)
- `data()` returns a **stable** `T*`

```
// fixed size, contiguous
std::array<int, 5> a{18, 21, 24, 27, 30};
// stable pointer
int *p = a.data();

// unchecked
a[1] = 22;
// bounds-checked
int v = a.at(2);

// constexpr
auto n = a.size();
for (int x: a) {
    /* use x */
}
```

stdlib containers:

`std::set`

- If you need **unique keys, sorted traversal and stable iterators**, use `std::set`
- Sorted by comparator (default **`std::less<T>`**)
- **Usually implemented as Red-black trees: not contiguous in memory but node-based** (one allocation per element)
- `insert` returns `{iterator, bool inserted}`

```
// fixed size, contiguous
std::array<int, 5> a{18, 21, 24, 27, 30};
// stable pointer
int *p = a.data();

// unchecked
a[1] = 22;
// bounds-checked
int v = a.at(2);

// constexpr
auto n = a.size();
for (int x: a) {
    /* use x */
}
```

stdlib containers:

`std::set`

- `insert`, `find`, `erase`, `lower_bound`, `upper_bound`:
 $O(\log N)$
- `count(k)` returns 0 or 1 (also
 $O(\log N)$)
- **Invalidation:** `insert` keeps all existing iterators/references **valid**;
`erase` invalidates **only** iterators to the erased element
- If ordering isn't needed →
`unordered_set` (avg $O(1)$)

```
// fixed size, contiguous
std::array<int, 5> a{18, 21, 24, 27, 30};
// stable pointer
int *p = a.data();

// unchecked
a[1] = 22;
// bounds-checked
int v = a.at(2);

// constexpr
auto n = a.size();
for (int x: a) {
    /* use x */
}
```

stdlib containers:

`std::unordered_set`

- If you need **unique keys**, **average $O(1)$ lookups**, and **no ordering**, use `std::unordered_set`
- **Unordered hash table**: no sorted traversal; elements grouped in **buckets**
- **Not contiguous** in memory; **node-based** elements + a **bucket array**

```
// Basic usage
std::unordered_set<int> us;
// plan buckets → fewer rehashes
us.reserve(1024);
us.insert(30);

bool ok = us.contains(30);
us.erase(30);

// tune density
us.max_load_factor(0.7f);
```

stdlib containers:

`std::unordered_set`

- **Insert may rehash** → **invalidates all iterators**
- **Erase** invalidates iterators to the erased element only
- **References/pointers to elements remain valid** (node-based), unless erased

```
// Basic usage  
std::unordered_set<int> us;  
// plan buckets → fewer rehashes  
us.reserve(1024);  
us.insert(30);  
  
bool ok = us.contains(30);  
us.erase(30);  
  
// tune density  
us.max_load_factor(0.7f);
```

stdlib containers:

`std::unordered_set`

Compared to `std::set`:

- **Ordering:** `unordered` vs sorted by comparator
- **Complexity:** avg $O(1)$ vs $O(\log N)$
- **Iterators:** may invalidate on rehash vs stable on insert
- **APIs:** no `lower_bound/upper_bound` vs available
- **Memory:** bucket array overhead; often **higher memory**, faster lookups

```
// Basic usage
std::unordered_set<int> us;
// plan buckets → fewer rehashes
us.reserve(1024);
us.insert(30);

bool ok = us.contains(30);
us.erase(30);

// tune density
us.max_load_factor(0.7f);
```

stdlib containers:

`std::unordered_set`

Compared to `std::set`:

- **Ordering:** `unordered` vs sorted by comparator
- **Complexity:** avg $O(1)$ vs $O(\log N)$
- **Iterators:** may invalidate on rehash vs stable on insert
- **APIs:** no `lower_bound/upper_bound` vs available
- **Memory:** bucket array overhead; often **higher memory**, faster lookups

```
// Basic usage
std::unordered_set<int> us;
// plan buckets → fewer rehashes
us.reserve(1024);
us.insert(30);

bool ok = us.contains(30);
us.erase(30);

// tune density
us.max_load_factor(0.7f);
```

stdlib containers:

`std::unordered_map`

- If you need **unique keys**, **average $O(1)$ lookups**, and **no ordering**, use `std::unordered_map`
- **Unordered hash table**: no sorted traversal; elements grouped in **buckets**
- **Not contiguous** in memory; **node-based** elements + a **bucket array**

```
// Basic usage
std::unordered_map<std::string, int> score;
// fewer rehashes as elements are added
score.reserve(1024); // reserving space

// inserts default value then assigns
score["Bob"] = 30;
score["Bob"] += 1; // -> 31

bool has = score.contains("Alice"); // C++20

score.try_emplace("Alice", 28);
// insert only if missing; avoids extra moves
score.emplace("Carla", 29); // constructs in-place
score.erase("Bob");

// Unordered traversal (arbitrary order)
for (const auto& [k, v] : score) { /* ... */ }
```

stdlib containers:

`std::unordered_map`

- `operator[]` **inserts** a default value when missing; `.at(k)` **throws** if missing; `find`, `contains` (C++20)
- Prefer `emplace` / `try_emplace` to avoid extra default constructions and **double lookups**
- `insert`, `find`, `erase`, `contains`: **avg** $O(1)$, **worst** $O(N)$

```
// Basic usage
std::unordered_map<std::string, int> score;
// fewer rehashes as elements are added
score.reserve(1024); // reserving space

// inserts default value then assigns
score["Bob"] = 30;
score["Bob"] += 1; // -> 31

bool has = score.contains("Alice"); // C++20

score.try_emplace("Alice", 28);
// insert only if missing; avoids extra moves
score.emplace("Carla", 29); // constructs in-place
score.erase("Bob");

// Unordered traversal (arbitrary order)
for (const auto& [k, v] : score) { /* ... */ }
```

stdlib containers:

std::map

- If you need **unique keys, ordered traversal by key, and stable iterators**, use std::map
- **Sorted by comparator** (default **std::less<Key>**); in-order traversal by key
- **Not contiguous** in memory; **node-based balanced tree** (one allocation per element)

```
std::map<std::string, int> m;  
  
m["Bob"] = 30;  
// inserts default 0, then assigns 30  
  
m.insert({"Alice", 28});  
// or: m.emplace("Alice", 28)  
m.try_emplace("Carla", 29);  
// constructs mapped only if key absent  
m.insert_or_assign("Bob", 31);  
// insert if missing, else assign (keeps key)  
  
auto it = m.find("Alice"); // O(log N)  
bool has = m.contains("Eve"); // C++20  
  
// Ordered traversal by key  
for (const auto& [k, v] : m) { /* ... */ }
```

stdlib containers:

std::map

- value_type is **std::pair<const Key, T>** → **key is const**
- insert, emplace, find, erase, lower_bound, upper_bound: **$O(\log N)$**
- operator[]: $O(\log N)$ (+ construction/assignment of mapped value)
- If ordering isn't needed → **std::unordered_map** (avg $O(1)$)

```
std::map<std::string, int> m;  
  
m["Bob"] = 30;  
// inserts default 0, then assigns 30  
  
m.insert({"Alice", 28});  
// or: m.emplace("Alice", 28)  
m.try_emplace("Carla", 29);  
// constructs mapped only if key absent  
m.insert_or_assign("Bob", 31);  
// insert if missing, else assign (keeps key)  
  
auto it = m.find("Alice"); // O(log N)  
bool has = m.contains("Eve"); // C++20  
  
// Ordered traversal by key  
for (const auto& [k, v] : m) { /* ... */ }
```

stdlib containers:

insert **vs** emplace

- **insert(value)**: you **build a value_type first**, then the container **copies/moves** it into place
- **emplace(args...)**: the container **constructs the element in place** using forwarded **constructor arguments (avoids a temporary)**
- **Same complexity & invalidation** as their positional variants
- Benefit of emplace: mainly when constructing **non-trivial objects** (e.g., `std::string`, `std::vector`, custom types)

stdlib containers:

`std::string`

- If you need an **owning mutable byte string** with **contiguous storage** and **dynamic growth**, use `std::string`
- Stores **bytes (not Unicode code points)**; `size()` counts **bytes**
- **Contiguous**; `c_str()` / `data()` return **NUL-terminated** pointer
- **Small-String Optimization (SSO)**: short strings stored **inline** (size & threshold are **implementation defined**)

```
std::string s = "Hi";  
// plan capacity → fewer reallocations  
s.reserve(16);  
  
// size = 3  
s.push_back('!');  
  
// NUL-terminated; writable since C++17  
auto p = s.data();  
s += " Marco";           // append  
  
s.insert(2, " there");   // O(n) shift  
s.erase(2, 6);          // remove " there"  
  
bool starts = s.starts_with("Hi"); // C++20  
  
// SSO vs heap (impl-defined thresholds)  
// likely SSO (inline)  
std::string a = "123456789012345";
```

stdlib containers:

`std::string`

- **Lexicographic operators:** `==`, `!=`, `<`, `<=`, `>`, `>=` (byte-wise)
- **`compare()`** returns `< 0 / 0 / > 0` (lexicographic)
- **Mixed types:** compare with **string literals / `const char*` / `std::string_view`** (no copy when using `string_view`)
- **Case-insensitive ops are not built-in:** use `tolower/locale` or compare lowercased views

```
std::string a = "abc", b = "abd";
bool eq = (a == b);    // false
bool lt = (a < b);     // true (lexicographic)

int cmp = a.compare(b); // < 0

std::string s = "Hello Marco";

bool p = s.starts_with("Hello"); // true
bool q = s.ends_with("co");      // true

auto pos = s.find("Marco");      // index or npos
std::string t = s.substr(6, 5);  // "Marco" (copy)

int x = std::stoi("42");         // 42
std::string y = std::to_string(123); // "123"
```

stdlib containers:

std::string

- **Prefix/Suffix:** starts_with, ends_with (**C++20**)
- **Search:** find, rfind, find_first_of, find_first_not_of, find_last_of, ...
- **Substrings:** substr(pos, count) (returns a **copy**); use std::string_view for **no-copy** views
- **Append/Build:** operator+=, append, push_back (amortized **O(1)**; reallocation moves bytes)
- **Capacity:** reserve, capacity, shrink_to_fit() (*non-binding*)
- **Conversions:** std::stoi, stol, stoll, stof, stod, stold; std::to_string
- **Formatting:** std::format (**C++20**) for safe string formatting

```
std::string a = "abc", b = "abd";
bool eq = (a == b);    // false
bool lt = (a < b);     // true (lexicographic)

int cmp = a.compare(b); // < 0

std::string s = "Hello Marco";

bool p = s.starts_with("Hello"); // true
bool q = s.ends_with("co");      // true

auto pos = s.find("Marco");      // index or npos
std::string t = s.substr(6, 5);  // "Marco" (copy)

int x = std::stoi("42");         // 42
std::string y = std::to_string(123); // "123"
```

C++ classes:

`noexcept` and move semantics

- Move operations transfer ownership of resources
- If a move operation **throws**, the source object may be left in a **partially moved state** (invalid or inconsistent)
- This breaks exception safety guarantees
- If move can throw, you cannot guarantee **strong exception safety**

```
struct Widget {  
    char* data;  
    Widget(Widget&& other) {  
        data = other.data;  
        other.data = nullptr;  
        if (/* some condition */)   
            throw std::runtime_error("fail");  
    }  
};
```

Problem:

- `other.data` is already set to `nullptr`
- Exception occurs → target object is not fully constructed, source object is empty
- State is **corrupted**: original resource is lost → leak or undefined behavior

C++ classes: `noexcept`, move semantics and standard containers

- **Standard containers** (like `std::vector`) need to **reallocate and move elements** when growing
- They want **strong exception safety**
- **If move is not `noexcept`, containers fall back to copy, which is safer but slower**

```
struct Widget {  
    char* data;  
  
    Widget(Widget&& other) noexcept {  
    }  
  
};
```

Problem:

- `other.data` is already set to `nullptr`
- Exception occurs → target object is not fully constructed, source object is empty
- State is **corrupted**: original resource is lost → leak or undefined behavior

C++ classes: destructor are called automatically when...

1. a local-object (stack-allocated) goes out of scope
2. *delete* is used on a pointer to a heap-allocated object
3. a temporary object is destroyed. For example, after an expression ends:...
4. a base class or member object is destroyed. In inheritance or composition, destructors are called in reverse order of construction
5. a container (e.g., `std::vector`) holding objects is destroyed, the destructor of each element is called automatically

```
struct Tracer {  
    const char* name;  
    explicit  
    Tracer(const char* n) noexcept : name(n) {}  
    ~Tracer() {  
        std::cout << "Destroy " << name << "; ";  
    }  
};  
  
int main() {  
    {  
        std::vector<Tracer> v;  
        // optional: preallocate space to  
        // avoid reallocations  
        v.reserve(2);  
  
        v.emplace_back("A");  
        v.emplace_back("B");  
    } // end of scope → vector destroyed  
        → elements destroyed  
}
```

C++ classes: destructor are called automatically when...

1. a local-object (stack-allocated) goes out of scope
2. *delete* is used on a pointer to a heap-allocated object
3. a temporary object is destroyed. For example, after an expression ends:...
4. a base class or member object is destroyed. In inheritance or composition, destructors are called in reverse order of construction
5. a container (e.g., `std::vector`) holding objects is destroyed, the destructor of each element is called automatically

```
struct Tracer {  
    const char* name;  
    explicit  
    Tracer(const char* n) noexcept : name(n) {}  
    ~Tracer() {  
        std::cout << "Destroy " << name << " ";  
    }  
};  
  
int main() {  
    std::vector<std::unique_ptr<Tracer>> v;  
    v.emplace_back(std::make_unique<Tracer>("A"));  
    v.emplace_back(std::make_unique<Tracer>("B"));  
  
    // vector destroyed → each unique_ptr  
    // destroyed → heap objects deleted  
    // → ~Tracer() called  
}
```